

Pageflow

How the client connects to the server — data flow, request lifecycle & internals

Pageflow is the front-end SPA engine of the AlfacodeTeam PhpServicePlatform. It is a fork of **Inertia.js v2**, rebranded and wired into the platform's PHP kernel. This guide explains *how the pieces connect* so you can read, debug, and contribute to the code with confidence.

Who this is for: new contributors / interns who have never touched Inertia or Pageflow.

What you'll learn: the mental model, the request lifecycle end-to-end, the wire protocol (HTTP headers), the file map, and how to trace a click from the browser to the PHP controller and back.

Location: `plugins/Pageflow/ui/` • Adapter: React • Core: TypeScript • Transport: `axios`

1. The Mental Model — What problem does Pageflow solve?

A traditional website does a **full browser reload** on every click: the server returns a whole HTML page, the browser throws away everything and re-parses. This is simple but feels slow.

A traditional SPA (React Router + REST API) feels fast but forces you to build **two apps**: a JSON API on the server and a client-side router + data-fetching layer in the browser. That's a lot of duplicated routing, validation, and state.

Pageflow is the middle path. You keep writing normal server-side controllers, but instead of returning HTML, they return a small **JSON "page object"**. Pageflow's client intercepts link clicks and form submits, fetches that JSON in the background, and swaps the React component in place — no full reload.

One sentence to remember: Pageflow turns your server responses into a stream of JSON "page objects," and the client swaps React components to match — giving you an SPA feel while you write plain server controllers.

The "page object" — the heart of the protocol

Every Pageflow response is a JSON object shaped like this (see `core/types.ts` → `Page`):

```
{
  "component": "Users/Index",           // which React component to render
  "props": { "users": [ ... ] },        // the data your controller sent
  "url": "/users",                      // the URL shown in the address bar
  "version": "a1b2c3",                 // asset version (cache-busting)
  "clearHistory": false,
  "encryptHistory": false
}
```

The client's whole job is: **take a page object** → **find the component named by `component`** → **render it with `props`** → **update the address bar to `url`**. Everything else (prefetch, polling, deferred props, forms) is sugar on top of this loop.

2. The Two Halves: `core/` and `react/`

The `ui/` folder splits cleanly into a framework-agnostic engine and a thin React binding.

Folder	Role	Depends on React?
<code>core/</code>	The engine: routing, HTTP, history, scroll, prefetch, polling, encryption. Pure TypeScript.	No
<code>react/</code>	The binding: <code><Link></code> , <code>useForm</code> , <code>usePage</code> , <code><Head></code> , the root <code>App</code> component.	Yes
<code>index.ts</code>	Helper <code>resolvePageComponent()</code> — maps a component name string to a file.	No

Why the split matters: the same `core/` engine could drive a Vue or Svelte binding. When you fix a bug in navigation, history, or the wire protocol, it lives in `core/`. When you fix something about how React renders, it lives in `react/`.

The single global instance

All of `core/` is coordinated by **one** `Router` instance, created once in `core/index.ts`:

```
export const router = new Router()
```

The React layer re-exports this same object. When you call `router.get('/users')` or `useForm().post('/login')`, you are talking to this one shared router.

3. Startup — How the app boots

The PHP server renders *one* real HTML page on the first visit. It contains a root `<div id="app" data-page="{...json...}">` where `data-page` is the initial page object. From there, the browser never does a full reload again.

The boot sequence (`react/createPageflowApp.tsx`)

1. Read the root element and parse `data-page` → the **initial page object**.
2. Resolve the initial component (e.g. load the `Users/Index` React file).
3. Decrypt history state if needed (`router.decryptHistory()`).
4. Mount the React `<App>` component, which calls `router.init(...)`.
5. `router.init()` wires up: the current page store, browser Back/Forward listeners, and scroll tracking.
6. Start the top-of-page progress bar (`setupProgress`).

```
// Your project's entry file (main.tsx) calls this once:
createPageflowApp({
  resolve: (name) => resolvePageComponent(name, import.meta.glob('./Pages/**/*.tsx')),
  setup({ el, App, props }) {
    createRoot(el).render(<App {...props} />)
  },
})
```

The `App` component (`react/App.tsx`) holds the "current page" in React state. Its one job is to swap that state whenever the router hands it a new page object — which re-renders the right component.

4. The Request Lifecycle — a click, end to end

This is the most important diagram in the guide. Follow a user clicking a `<Link href="/users">` :



```
| yes → setPage()
▼
6. currentPage.set(page)    core/page.ts — Architecture & Flow Guide
|   • resolve component "Users/Index"
|   • push new URL into browser history
|   • fire 'navigate' event
▼
7. App.swapComponent()    react/App.tsx
|   React re-renders <Users/Index> with new props
▼
8. Scroll reset, progress bar hides. Done – no full reload.
```

The magic is step 4 ↔ 5. The server and client agree on a tiny contract: requests carry an `X-Pageflow: true` header, and Pageflow responses echo `X-Pageflow: true` back. If that header is missing on the response, the client knows it got a "normal" page (e.g. a login redirect to an external site) and does a full browser navigation instead.



5. The Wire Protocol — HTTP headers

This is the exact contract your PHP side must speak. All headers were renamed from Inertia's `X-Inertia-*` to `X-Pageflow-*` during the fork. Set on the **request** by `core/http/request.ts` and `requestParams.ts`:

Header	Direction	Meaning
<code>X-Pageflow: true</code>	request & response	"This is a Pageflow exchange." The response MUST echo it or the client treats it as a hard navigation.
<code>X-Pageflow-Version</code>	request	Client's asset version. If the server's version differs, it forces a full reload to pick up new JS/CSS.
<code>X-Pageflow-Partial-Component</code>	request	Only reload data for THIS component (partial reload guard).
<code>X-Pageflow-Partial-Data</code>	request	Comma list of prop keys to include (the <code>only</code> option).
<code>X-Pageflow-Partial-Except</code>	request	Comma list of prop keys to exclude (the <code>except</code> option).
<code>X-Pageflow-Reset</code>	request	Prop keys whose merged/paginated state should be reset.
<code>X-Pageflow-Error-Bag</code>	request	Namespace for validation errors (multiple forms on a page).
<code>X-Pageflow-Location</code>	response (409)	External redirect. Server replies <code>409</code> + this header; client does a real browser navigation.

The redirect trick

Browsers follow `302` redirects transparently in `fetch/XHR`, which would break SPA navigation. So for an **external** redirect, the server responds `409 Conflict` with `X-Pageflow-Location`, and the client (`response.ts` → `locationVisit()`) performs a real `window.location` navigation. For *internal* redirects the server just returns the new page object directly.

6. File Map — where everything lives

core/ — the engine

File	What it does
<code>router.ts</code>	The public API: <code>get/post/put/patch/delete/visit/reload/poll/prefetch</code> . Orchestrates everything.
<code>http/request.ts</code>	Fires the actual <code>axios</code> call, attaches headers, handles cancellation & progress.

<code>http/response.ts</code>	Interprets the response: is it a page? merge props? errors? redirect? Then updates the page.
<code>http/requestParams.ts</code>	Normalizes visit options into headers & query/body data; records callbacks.
<code>http/requestStream.ts</code>	Queues concurrent requests; cancels/interrupts in-flight ones.
<code>page.ts</code>	The "current page" store. Resolves components, swaps them, fires page events.
<code>history.ts</code>	Reads/writes <code>window.history</code> ; saves scroll positions & remembered state.
<code>encryption.ts</code>	AES-GCM encrypt/decrypt of history state (privacy on Back button).
<code>scroll.ts</code>	Saves & restores scroll positions per region and for the document.
<code>prefetched.ts</code>	Cache of pre-fetched pages (hover-to-preload).
<code>poll/</code>	Interval polling that pauses in background tabs.
<code>eventHandler/</code>	Global event bus (<code>pageflow:start</code> , <code>:success</code> , <code>:error</code> ...) + Back/Forward + scroll listeners.
<code>url.ts / formData.ts / files.ts</code>	URL building, query-string merging, FormData conversion for file uploads.
<code>server.ts</code>	Optional Node SSR server (renders the first page for SEO).

react/ — the binding

File	What it does
<code>App.tsx</code>	Root component. Holds current page in state; swaps components on navigation.
<code>createPageflowApp.tsx</code>	Boot function (CSR + SSR) you call once in your entry file.
<code>Link.tsx</code>	The smart <code><a></code> . Intercepts clicks, supports prefetch-on-hover, method, data.
<code>useForm.tsx</code>	Form state + submit + validation errors + progress. The most-used hook.
<code>usePage.tsx</code>	Read the current page's props/shared data inside a component.
<code>Head.tsx</code>	Set <code><title></code> and meta tags per page.
<code>Deferred / WhenVisible / usePoll / usePrefetch / useRemember</code>	Optional UX features (lazy props, on-scroll load, polling, prefetch, persisted local state).

7. The Features (and which file to open)

Everyday (learn these five first)

You want to...	Use	File
Link between pages	<code><Link href></code>	react/Link.tsx
Read controller data	<code>usePage()</code>	react/usePage.tsx
Build a form + show errors	<code>useForm()</code>	react/useForm.tsx
Set the page title	<code><Head></code>	react/Head.tsx
Navigate from code	<code>router.get()/post()</code>	core/router.ts

Performance & UX sugar (add later)

Feature	What it does
Partial reloads (<code>only</code> / <code>except</code>)	Re-fetch just some props instead of the whole page.
Deferred props	Render the page now, load slow data (e.g. a report) right after.
WhenVisible	Load data only when an element scrolls into view (infinite scroll).
Prefetch	Fetch a page before the click (e.g. on hover) → opens instantly.
Polling	Auto re-fetch every N seconds (dashboards). Pauses in background tabs.
Remember	Keep local state (half-filled form) across navigation.
History encryption	Encrypt cached page data in browser history; wipe on logout.
Merge / deepMerge props	Append paginated data to existing props (match by key).

The form flow (most common contribution area)

```
const form = useForm({ email: '', password: '' })

// form.data.email, form.setData('email', v), form.errors, form.processing ...

form.post('/login') // → router.post → Request.send → Response.handle
// on 422 with errors → form.errors is populated, form.hasErrors = true
// on success page object → component swaps, form.wasSuccessful = true
```

Internally `useForm` wraps the router's visit callbacks (`onStart` , `onSuccess` , `onError` , `onFinish`) to drive `processing` , `errors` , and `recentlySuccessful` state. Validation errors arrive as the `errors` prop on the page object.

8. How to Contribute — a practical map

"I want to change how navigation works"

Pageflow — Architecture & Flow Guide

Start in `core/router.ts` → `visit()`. Follow the `Request` → `Response` classes. Behavioral navigation state lives in `core/page.ts` and `core/history.ts`.

"I want to change the wire protocol / headers"

`core/http/request.ts` (request headers) and `core/http/requestParams.ts` (partial-reload headers). The response side reads them in `core/http/response.ts`. Remember: **any header change must be mirrored on the PHP side** (`plugins/Pageflow/Http/`).

"I want to change React rendering / add a hook"

Everything is in `react/`. Add your hook there, export it from `react/index.tsx`. Read the current page with `usePage()`; trigger navigation with the exported `router`.

Golden rule for this fork: the `core/` engine is a faithful copy of Inertia.js v2. Before "fixing" something there, check whether it's intentional upstream behavior. Keep `core/` framework-agnostic — no React imports, no business logic. Protocol changes are the one place client and server are tightly coupled, so change both sides together.

Debugging tips

- Open DevTools → Network. A Pageflow request shows `X-Pageflow: true` and returns JSON, not HTML.
- Listen to global events for tracing: `router.on('start' | 'success' | 'error' | 'finish', cb)`.
- If a click causes a *full reload* instead of an SPA swap, the response was missing `X-Pageflow` or the asset `version` changed — check the response headers first.
- Validation not showing? Confirm the server returns an `errors` prop and (if scoped) the matching `X-Pageflow-Error-Bag`.

Glossary

Term	Meaning
Page object	The JSON <code>{component, props, url, version}</code> returned by the server.
Visit	One navigation request with its options (method, data, only/except, callbacks).
Partial reload	A visit that only fetches some props (via <code>only</code> / <code>except</code>).
Shared props	Data the server attaches to every page (e.g. the logged-in user).
Deferred prop	A prop the server sends in a second request after the page renders.
Error bag	Namespace for validation errors when a page has multiple forms.

9. Server Integration — the PHP side

Everything above is the browser half. The PHP half lives in `plugins/Pageflow/` (NOT under `ui/`) and is a normal GDA plugin (`solves: "http.pageflow"`). Its job is the mirror image of the client: take a controller's *component name* + *props* and emit either the JSON page object (for an XHR navigation) or the full HTML shell (for a first load).

The pieces

File	Role
<code>Http/PageflowResponder.php</code>	The heart. <code>render(\$request, \$component, \$props)</code> negotiates JSON vs HTML, merges shared props, applies partial reloads, and injects the CSRF token.
<code>Http/PageflowPage.php</code>	The page-object value type. <code>toArray()</code> → JSON; <code>mount(\$appId)</code> → the <code><div data-page></code> boot element.
<code>Http/PageflowVersionStage.php</code>	Pipeline stage. On a stale <code>X-Pageflow-Version</code> (GET), replies <code>409</code> + <code>X-Pageflow-Location</code> so the client hard-reloads new assets.
<code>Http/PageflowShareStage.php</code>	Runs registered <code>pageflow_share()</code> contributors into the responder each request.
<code>Provider.php</code>	Wires the responder (per-request singleton), the CSRF resolver, and the two pipeline hooks.
<code>resources/views/app.php</code>	Sample root layout (the HTML shell). Pointed to by <code>PAGEFLOW_ROOT_VIEW</code> .

How a controller uses it

Controllers stay thin — they inject the responder contract and return `render()` :

```
final class UserController
{
    public function __construct(
        private readonly PageflowResponder $pageflow,
        private readonly UserServiceContract $users,
    ) {}

    public function index(Request $request): Response
    {
        return $this->pageflow->render(
            $request,
            'Users/Index', // the React component name
            ['users' => $this->users->all()], // the props
        );
    }
}
```

That single call returns JSON to a Pageflow XHR and the HTML shell to a first-time browser hit — the controller doesn't care which.

Pageflow — Architecture & Flow Guide

Request → decision → response (server view)



Shared props — data on every page

Register once at bootstrap; resolved per request. This is how the logged-in user, app name, locale, etc. reach every page without repeating them in each controller:

```
// keyed share – resolver receives the current Request
pageflow_share('auth', fn($request) => $request->identity()?->userId);
pageflow_share('appName', fn() => 'HKM');

// on the client: const { auth, appName } = usePage().props;
```

CSRF — how the two halves connect

This is the seam that makes **forms** work. Trace it once and you'll never be confused by a 403 again:



- The token is minted from `APP_KEY` + the raw session-cookie value (the binding). Config: `PAGEFLOW_CSRF_COOKIE` (default `hkm_session`) and `PAGEFLOW_CSRF_LIFETIME` — the lifetime **must match** the `CsrftokenLayer` in your `withSecurity([...])`.
- The bound cookie must be in the Cookie plugin's `encrypt_exempt` list, or the binding won't match the raw value the layer reads.
- Empty `APP_KEY` ⇒ no token minted (safe for GET-only apps).

The root layout (`resources/views/app.php`)

Rendered in an **isolated scope** by the responder via `ob_start()` — no globals leak in. Only three variables are available:

Variable	Type	Use
<code>\$FLOW_PAGE</code>	PageflowPage	<code>\$FLOW_PAGE->mount(\$FLOW_APP_ID)</code> emits the boot <code><div data-page></code> .
<code>\$FLOW_CSRF</code>	string	The token for the <code><meta name="csrf-token"></code> tag (<code>'</code> if none).
<code>\$FLOW_APP_ID</code>	string	Root element id (default <code>app</code>).

```
<!DOCTYPE html>
<html lang="<?= $locale ?>">
<head>
  <meta charset="utf-8">
  <!-- client reads this, sends it back as X-CSRF-Token -->
  <?php if ($FLOW_CSRF !== ''): ?>
    <meta name="csrf-token" content="<?= htmlspecialchars($FLOW_CSRF, ENT_QUOTES) ?>">
  <?php endif; ?>
  <title><?= $pageTitle ?></title>
  <script type="module" src="/build/app.js?v=<?= $assetVersion ?>" defer></script>
</head>
<body>
  <!-- the ONE line that matters: emits <div id="app" data-page="{json}"> -->
  <?= $FLOW_PAGE->mount($FLOW_APP_ID) ?>
</body>
</html>
```

The #1 gotcha: the client boots from the root element's `data-page` attribute (`JSON.parse(el.dataset.page)`), *not* from `window.initialPage`. Always emit the mount point with `$FLOW_PAGE->mount(...)`. The legacy `renderScript()` / `window.initialPage` path is deprecated and will crash the current client.

Config summary (`.env`)

Key	Default	Meaning
<code>PAGEFLOW_VERSION</code>	<code>1</code>	Asset version / build hash. Bump it to force clients onto new JS/CSS.

<code>PAGEFLOW_ROOT_VIEW</code>	<code>''</code>	Absolute path to the layout. Empty ⇒ built-in minimal document.
<code>PAGEFLOW_APP_ID</code>	<code>app</code>	Root element id the client mounts into.
<code>PAGEFLOW_CSRF_COOKIE</code>	<code>hkm_session</code>	Session cookie the CSRF token binds to.
<code>PAGEFLOW_CSRF_LIFETIME</code>	<code>43200</code>	Token lifetime (s). Must match the CsrfTokenLayer.

End-to-end round trip (both halves)

Step	Client expects	Server provides
First load boot	<code>JSON.parse(el.dataset.page)</code>	<code><div data-page></code> via <code>mount()</code>
CSRF on mutations	reads <code><meta csrf-token></code> → <code>X-CSRF-Token</code>	renders the meta + mints a matching token
XHR detection	<code>x-pageflow</code> header on response	<code>X-Pageflow: true</code>
Stale assets	<code>409</code> + <code>x-pageflow-location</code>	<code>PageflowVersionStage</code>
Partial reload	<code>Partial-Data/-Except/-Component</code>	<code>resolvePartial()</code>
Shared props	<code>usePage().props</code>	<code>pageflow_share()</code> + share stage

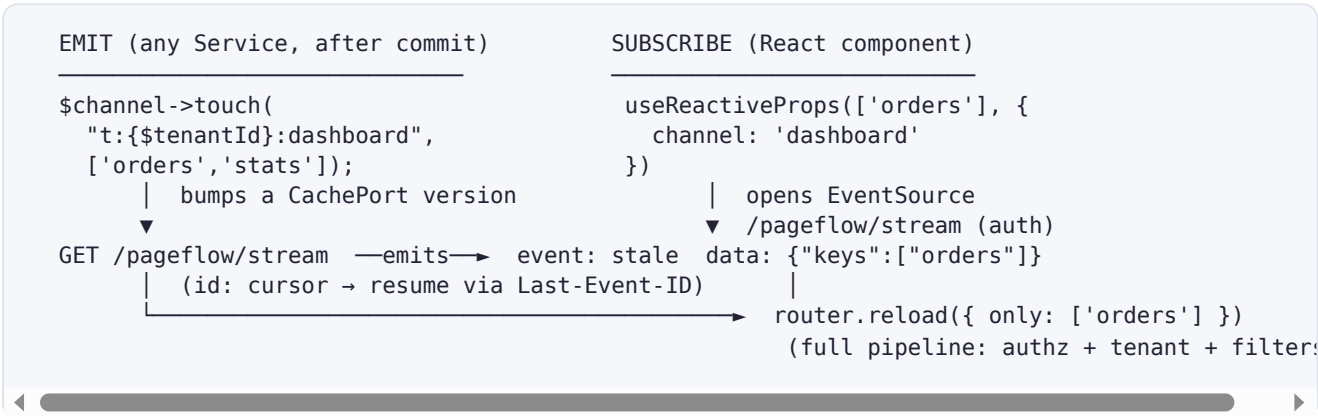
10. Beyond Inertia — the platform-native capabilities

These features exist in Pageflow but **not** in Inertia.js, because they lean on things the AlfaCodeTeam kernel owns (OpenSwoole, EventBus, **Identity**, ports, multi-tenancy). This is the part that makes Pageflow *yours*, not just a rebrand.

The security spine running through all of it: realtime pushes only *signals*, never data; every actual byte still flows through the authenticated pipeline. Client-side permission checks are UX only — the Service layer stays the authority.

10.1 Reactive props (secure server push)

Live data with **zero polling**. The server pushes "these prop keys are stale"; the client does a normal authenticated partial reload. A hijacked channel leaks nothing.



- **Emit:** inject `PageflowChannel` into a Service and call `touch()` after commit — or subscribe it to an integration event on the EventBus. It bumps a `CachePort` version (cheap), it does not scan a DB.
- **Endpoint:** `GET /pageflow/stream` ships ready-made (`PageflowEndpointsController`), auth-gated, tenant-scoped. The client picks a topic; the server prefixes the tenant so cross-tenant watching is impossible.
- **Runtime:** SSE holds a worker — use OpenSwoole or a dedicated SSE pool, not a small FPM pool.

10.2 Native validation & precognition

No manual `share('errors')`, no separate Precognition library.

Flow	What happens
Auto error flow	A controller's DTO throws <code>ValidationException</code> ; <code>PageflowValidationStage</code> flashes errors to the session and 303-redirects back. The origin page re-renders (re-authorized, real props intact) and <code>useForm</code> shows <code>props.errors</code> . <code>preserveState</code> keeps the form.
Precognition	<code>Precognition: true</code> runs validation only. Client: <code>usePrecognition()</code> or <code><Form validateOn="blur"></code> → debounced live errors. Server: the controller short-circuits with <code>pageflow_precognition(\$request)</code> → <code>precognitionSuccess()</code> ; the stage flags the request so any layer can refuse writes.

```
// Live, server-truth validation — no rule duplication
const pre = usePrecognition('post /pageflow/register', {Flex: Data})


```

Harden precognition endpoints with the `throttle` route filter — it's a per-keystroke path.

10.3 Permission-aware UI

```
// server auto-shares a NON-SENSITIVE identity projection (configurable)
pageflow_auth_projection(fn($id) => ['canManage' => (bool) $id?->hasPermission('admin:manage')

// client — UX gating only; the Service layer is the real boundary
const { can, hasRole } = useAuth()
<Can permission="invoice:create"><NewInvoiceButton /></Can>
```

`useFlushOnIdentityChange()` at the app root wipes the prefetch cache on login/logout/tenant-switch — prevents a shared-tab data leak.

10.4 CSRF auto-refresh (long-lived SPAs)

The initial token expires after `PAGEFLOW_CSRF_LIFETIME`. Install an axios interceptor that, on a 403, fetches a fresh token from `GET /pageflow/csrf`, updates the `<meta>`, and replays the request once (single-flight — a burst of 403s triggers one refresh):

```
installCsrfAutoRefresh() // once at bootstrap
```

10.5 Performance & offline

- **Batched deferred props** — all deferred groups load in ONE partial reload instead of N (fewer round-trips).
- **Service worker** — `registerPageflowSW()` caches static assets (cache-first) and page objects (network-first, offline fallback). Never caches mutations. Call `clearPageflowSWCache()` on logout for shared devices.

10.6 End-to-end types (`hkm pageflow:types`)

Generates a `.d.ts` that merges the shared props into `PageProps` and enumerates every page component into a `PageflowPages` registry — so `usePage().props` is typed and adding a page gives autocomplete. Re-run after adding pages. Inertia treats this as out-of-scope; here it's one command.

Ready-made endpoints & new env

Route	Purpose	Auth
<code>GET /pageflow/csrf</code>	fresh CSRF token	session
<code>GET /pageflow/stream</code>	reactive SSE channel (tenant-scoped)	<code>auth</code> filter

Env	Default	Meaning
-----	---------	---------

PAGEFLOW_STREAM_INTERVAL	2000	SSE poll cadence (ms) — how often the stream checks the channel version
--------------------------	------	---

Pageflow Architecture & Flow Guide

Contributor rule for these features: keep the "push signals, pull data" invariant. If you ever make the stream send prop *values*, you've turned a safe channel into an authorization bypass. Emit key names; let the client re-fetch through the pipeline.

11. Security Hardening Ledger Platform Architecture & Flow Guide

The platform-native features were reviewed in several passes. This ledger records the controls now in place — treat them as invariants: if you touch this code, do not regress them.

Invariants (never break these)

- **Push signals, pull data.** The reactive channel emits prop *key names* only. Data is always re-fetched through the authenticated pipeline. Sending values would turn a safe channel into an authorization bypass.
- **CSRF token stays same-origin.** It is attached only to same-origin requests (`isSameOriginUrl` gate in the request builder, precognition, and the axios refresh interceptor) and lives only in the HTML `<meta>` tag + the throttled refresh endpoint — never in page-object JSON or the SW cache.
- **Client permission checks are UX only.** `useAuth` / `<Can>` hide UI; the Service layer is the authorization boundary.
- **Precognition never leaks data.** Responses carry field errors only.

Controls by surface

Surface	Control
Reactive SSE	Guest-blocked; tenant-scoped channel names; <code>id</code> : strips CR/LF (no stream injection); cache-key charset filter; bounded lifetime (<code>PAGEFLOW_STREAM_MAX_SECONDS</code> , default 300s) + <code>throttle:30,1</code> so a stream can't pin a worker.
CSRF	Same-origin-only attachment + credentials; token out of JSON/SW cache; single-flight refresh that fires ONLY on a CSRF-type 403 (not authz denials); <code>/pageflow/csrf</code> throttled.
Validation redirect	303 <code>Location</code> reduced to a same-origin path (referer & <code>X-Pageflow-Url</code>) — no open redirect (blocks <code>//host</code> , backslash, absolute URLs).
Service worker	Opt-in caching (secure by default): a page object is cached ONLY when the server sends <code>X-Pageflow-Cache: 1</code> (via <code>render(..., cacheable: true)</code>) or <code>Cache-Control: public</code> — authenticated pages carry neither, so a forgotten header can't leak data at rest. Never caches mutations or <code>no-store</code> / <code>private</code> ; scopes cache ops to <code>pageflow-*</code> (won't clobber other apps); purges directly from the page on identity change / logout.
Precognition	Debounced client + <code>throttle</code> ; request flagged (<code>precognition</code> attribute) so repos/services can refuse writes; cooperative short-circuit is the primary guard.
Auth share	Non-sensitive projection (never tokens); overridable via <code>pageflow_auth_projection()</code> to hide internal permission names.
Markup	All interpolation <code>htmlspecialchars</code> 'd; boots from a <code>data-page</code> attribute with NO inline script → CSP-friendly.

Accepted residual risks (by design — document, don't "fix" blindly)

Risk	Why it's acceptable / mitigation
------	----------------------------------

Precognition can execute writes if a controller forgets to short-circuit	Cooperative by design (like Laravel Precognition). The <code>precognition</code> attribute lets any layer refuse writes; opt-in <code>PAGEFLOW_PRECOGNITION_ROLLBACK</code> signals intent. We do NOT auto-wrap a transaction — that would collide with service-managed transactions.
Page objects cached at rest by the SW	Now OPT-IN — only pages the developer explicitly marks <code>cacheable</code> (public, no user data) are stored; authenticated pages are never cached by default. Network-first, purged on identity change.
Reactive stale <i>key names</i> visible to authorized tenant users	Names only, tenant-scoped, no data. Avoid encoding secrets in prop key names.
Validation enumeration via precognition	App-level: validation messages must not reveal resource existence; apply <code>throttle</code> and don't log precognition bodies (may contain passwords).
User's own permission list shared to their browser	Standard; it's their own capabilities. Use <code>pageflow_auth_projection()</code> for capability booleans if the naming is sensitive.

Operational reminders: run the SSE stream under OpenSwoole (not a small FPM pool); keep the CSRF bound cookie in Cookie's `encrypt_exempt` ; the auto-error flow wants the Session plugin (degrades to a 422 envelope otherwise).

Pageflow Contributor Guide · generated for the AlfacodeTeam PhpServicePlatform · source: `plugins/Pageflow/` (PHP) + `plugins/Pageflow/ui/` (client). Pageflow is a fork of Inertia.js (MIT) — retain upstream attribution.